Single-Trait Models

Peter Sørensen

Table of contents

The single-trait Bayesian linear regression (ST-BLR) layer is the common statistical foundation for the direct BED, summary-statistics CSR, scheduled sparse-update, and annotation-informed workflows in `sblr`. These interfaces use different data representations and prior refinements, but they share the same basic objective: estimate sparse marker effects for one phenotype while accounting for linkage disequilibrium (LD).

This note defines that core model, explains the quantities used by marker updates, and distinguishes statistical prior choices from computational scheduling controls. Data preparation and complete workflows are covered in Data representations and Sparse-LD and BED workflows.

1 Single-Trait Regression Model

For n individuals and m markers, let y be an n-vector of phenotypes and X an $n \times m$ genotype matrix. The single-trait linear model is

$$y = Xb + e,$$

where

- $b = (b_1, \ldots, b_m)^\top$ contains marker effects;
- $e \sim N(0, v_e I_n)$ is residual variation;
- v_e is the residual variance; and
- the genetic value $g = Xb$ has variance v_g.

When genotypes and phenotypes are centered, a realized genetic-variance quantity can be written as

$$v_g(b) = \frac{1}{n} b^\top X^\top X b.$$

This is distinct from the active marker-effect prior variance v_b. Informally, v_b controls the scale of nonzero effects before seeing the data, while v_g describes the aggregate variation generated by all marker effects and LD.

Although several ST-BLR wrappers accept multiple phenotype columns, their basic marker updates remain trait-specific. Running these updates for several traits is not automatically equivalent to a joint multi-trait model with effect or residual covariance. See Multi-trait models for that distinction.

2 Summary-Statistics Representation

The Gaussian regression calculations can be expressed using cross-products:

$$wy = X^\top y, \qquad XX = X^\top X, \qquad yy = y^\top y.$$

Let

$$w_i = x_i^\top x_i$$

be marker i's diagonal genotype cross-product. The marker-scale residual score vector is

$$r = X^\top (y - Xb) = wy - XXb.$$

For marker i, the partial residual score that adds its current effect back is

$$s_i = r_i + w_i b_i.$$

These quantities contain the information needed for a single-marker update: `wy` connects markers to the phenotype, XX propagates effect changes through LD, w_i controls the conditional precision, and `yy` contributes to residual-variance calculations and initialization. If b_i changes by Δb_i, the residual score can be updated without revisiting individual-level observations:

$$r_{\text{new}} = r_{\text{old}} - XX_{\cdot i}\Delta b_i.$$

`stblr_csr()` uses `stats$wy`, `stats$ww`, `stats$yy`, sample size n, and a disk-backed sparse approximation to XX. The complete sufficient- statistic and sparse-storage contracts are described in Technical summary statistics and Technical sparse LD and CSR.

3 Global BayesC-Style Sparse Prior

The baseline ST-BLR prior represents every marker as either null or active. Introduce an inclusion indicator

$$d_i \sim \text{Bernoulli}(\pi),$$

and write

$$b_i = d_i \alpha_i, \qquad \alpha_i \mid d_i = 1 \sim N(0, v_b).$$

Equivalently,

$$b_i \sim (1-\pi)\delta_0 + \pi N(0, v_b),$$

where δ_0 is a point mass at zero. The global parameters have different roles:

- π is the prior probability that a marker is active;
- v_b is the prior variance of an active marker effect;
- v_e is the residual variance;
- v_g is the genetic variance generated by the fitted marker effects.

The wrapper argument `pi_marker` is a backward-compatible initial active probability. Current wrappers separate related roles through `pi_init`, `pi_vb_init`, `pi_prior_mean`, and beta-prior controls. In particular, the initial active-effect variance is scaled from the initial heritability and expected number of active markers:

$$v_b^{(0)} \approx \frac{v_y h^2}{m\pi_{\text{vb}}},$$

where v_y is the phenotype variance and π_{vb} corresponds to `pi_vb_init`. This is an initialization and prior-scale choice, not a statement that exactly $m\pi_{\text{vb}}$ markers are causal.

3.1 Marker Inclusion Update

Under the global prior, the marginal log Bayes factor for marker i can be written as

$$\log BF_i = \frac{1}{2}\log\left(\frac{v_e}{v_e + w_i v_b}\right) + \frac{1}{2}\frac{s_i^2 v_b}{v_e(v_e + w_i v_b)}.$$

Combining this evidence with the prior odds gives

$$P(d_i = 1 \mid -) = \frac{\pi BF_i}{(1-\pi) + \pi BF_i}.$$

Conditional on inclusion, the marker effect has Gaussian full conditional

$$b_i \mid d_i = 1, - \sim N\left(\frac{s_i}{w_i + v_e/v_b}, \frac{v_e}{w_i + v_e/v_b}\right).$$

If excluded, $b_i = 0$. Repeated MCMC updates produce the posterior inclusion probability (PIP)

$$\text{PIP}_i = P(d_i = 1 \mid y, X, \text{prior}),$$

and posterior mean effect

$$\hat{b}_i = E(b_i \mid y, X, \text{prior}).$$

In fit objects these are generally returned as rows of `dm` and `bm`. PIPs quantify posterior evidence under the chosen data representation, LD, prior, and sampler. They are not probabilities that a marker is biologically causal.

4 BayesS-Style `selection_s` in CSR Models

The package argument `selection_s` corresponds to the BayesS-style MAF architecture parameter `S`. It is currently implemented only for the unscheduled CSR BayesC, CSR BayesR, and CSR SBayesRC backends. Scheduled CSR BayesC, BayesC-like annotation-aware CSR backends, and BED backends do not support sampler-level `selection_s`.

CSR effects are on the standardized-genotype scale. The MAF-dependent prior scale is

$$q_j(S) = h_j^{S+1}, \qquad h_j = 2p_j(1-p_j),$$

where p_j is the minor allele frequency. The `+1` exponent is used because the sampler effects are standardized-genotype-scale effects rather than allele-scale effects.

For CSR BayesC, fixed global `selection_s` changes the active-effect prior to

$$b_j \mid d_j = 1, v_b, S \sim N\{0, v_b q_j(S)\}.$$

For CSR BayesR, fixed global `selection_s` changes non-null component priors to

$$b_j \mid \text{component}_j = m, v_b, S \sim N\{0, v_b \gamma_m q_j(S)\}.$$

Sampled trait-specific `selection_s` is requested with:

```
fitC <- stblr_csr(
  stats = stats,
  Glist = Glist,
  method = "bayesC",
  estimate_selection_s = TRUE
)

fitR <- stblr_csr(
  stats = stats,
  Glist = Glist,
  method = "bayesR",
  estimate_selection_s = TRUE
)
```

The sampled-S defaults are `selection_s_init = 0`, `selection_s_prior = c(-3, 2)`, and `selection_s_proposal_sd = 0.35`. `selection_s = -1` gives `q_j(S) = 1`, so it reproduces the ordinary unscaled model under the same seed for fixed-S fits. See selection_s implementation status for the support matrix, MH kernels, and SBayesRC details.

5 Two Fitted Representations

5.1 Individual-Level BED: `stblr_bed()`

`stblr_bed()` is the high-level interface for fitting ST-BLR directly from PLINK BED markers referenced by a qgg `Glist`. It reads selected genotype columns during sampling and maintains individual-level residual information. A short illustrative call is:

```
fit_bed <- stblr_bed(
  Glist = Glist,
  y = y,
  method = "bayesC",
  chr = 1,
  cls = cls,
  pi_init = 0.001,
  h2 = 0.5,
  nit = 1000,
```

```
  nburn = 100
)
```

This path is appropriate when individual-level genotypes remain available and repeated BED access is acceptable. The statistical inputs include the phenotype, selected samples and markers, allele frequencies, and genotype scaling choice. A separately materialized LD matrix is not required because LD is present in the observed genotype columns.

5.2 Summary Statistics and CSR LD: `stblr_csr()`

`stblr_csr()` fits the same broad global-prior ST-BLR family using `make_stats()` output and disk-backed sparse LD metadata stored in `Glist$sparseLD` by `make_sparseLD()`:

```
fit_csr <- stblr_csr(
  stats = stats,
  Glist = Glist,
  method = "bayesC",
  pi_init = 0.001,
  h2 = 0.5,
  scheduled = FALSE,
  nit = 1000,
  nburn = 100
)
```

Here `stats$wy`, `stats$ww`, and `stats$yy` replace repeated phenotype and diagonal genotype calculations, while `Glist$sparseLD$prefix` supplies retained off-diagonal LD relationships. The CSR representation makes large sparse-LD analyses possible without loading dense XX into memory.

The two paths target closely related single-trait sparse-regression models when samples, markers, scaling, priors, and likelihood information are matched. They are not expected to produce identical Monte Carlo output. Thresholded sparse LD approximates the full genotype cross-products, and the implementations can differ in residual maintenance, update order, scheduling, floating-point behavior, and initialization. Finite MCMC variation adds further differences.

Use BED-versus-CSR agreement as an integration and sensitivity diagnostic. Compare broad posterior patterns, PIP rankings, effects, and variance summaries rather than expecting equality of individual draws.

6 Scheduled Sparse Updates and Runtime Controls

Most markers in a sparse model spend substantial time in the null state. Updating every null marker on every iteration can dominate runtime. Scheduled samplers reduce this work by revisiting low-priority null markers less often while retaining mechanisms that periodically reconsider them.

These settings are **computational controls, not definitions of a different prior**. They do not replace the BayesC-style π, v_b, or v_e parameters. They do, however, change the update schedule and

can affect finite-run behavior, mixing, and approximation quality. Aggressive skipping therefore requires convergence and sensitivity checks.

6.1 Common Scheduling Controls

Control	High-level role
`scheduled`	Selects the scheduled sampler path where supported.
`full_sweep_every`	Forces periodic reconsideration of all markers rather than only scheduled or active candidates.
`null_skip_base`	Sets a baseline interval for revisiting low-probability null markers in scheduled samplers.
`null_skip_max`	Caps adaptive null-marker skip intervals.
`candidate_threshold`	Treats markers with sufficiently large current inclusion probability as candidates for more frequent updates.
`candidate_lifetime`	Keeps candidates active for a number of scheduled sweeps after they were last considered interesting.
`skip_nulls_burnin_only`	Restricts null-marker skipping to burn-in when `TRUE`, so retained sampling uses the less aggressive post-burn-in schedule.

The BED non-scheduled sparse path also exposes `null_update_prob`, which controls probabilistic revisiting of null markers. BED samplers use `rebuild_every` to periodically rebuild individual-level residual state.

6.2 CSR LD-Neighbor Wake-Up

The scheduled CSR sampler additionally exposes:

- `wakeup_ld_neighbors`, which allows a marker update to reschedule retained LD neighbors;
- `wakeup_diff_threshold`, which requires a sufficiently large effect change before triggering neighbor wake-up; and
- `wakeup_max_neighbors`, which can cap the number of neighbors rescheduled.

The motivation is that changing one marker's effect changes the residual scores of correlated markers. Waking LD neighbors lets the schedule respond to that local change without immediately performing a complete sweep.

For validation, compare scheduled and unscheduled runs under matched priors and data, inspect sensitivity to full-sweep frequency and skip settings, and use sufficiently long retained sampling. Runtime improvement alone does not establish that posterior exploration is adequate.

7 Variance Updates and Interpretation

The public wrappers initialize marker-effect and residual variance structures from phenotype variance, `h2`, marker count, and inclusion-prior controls. During sampling, `updateB`, `updateE`, and `updatePi` determine whether the corresponding quantities are updated. The degrees-of-freedom controls `nub` and `nue` and associated scale matrices define their priors.

For one trait, covariance matrices reduce conceptually to scalar variance quantities, although formatted fits may retain matrix-shaped summaries for consistency with multi-trait code. Distinguish:

- the active marker-effect variance v_b;
- aggregate genetic variance v_g;
- residual variance v_e; and
- their MCMC traces and posterior summaries.

Initial `h2` helps set starting and prior scales. It is not guaranteed to equal the posterior or realized heritability. In simulations, differences can arise from finite data, realized architecture, shrinkage, sparse LD, and Monte Carlo variation.

8 Posterior Outputs and Diagnostics

Formatted ST-BLR fits are named lists. Components depend on the wrapper and sampler path, so inspect `names(fit)` and `fit$input` before interpreting them.

Component	Single-trait interpretation
`bm`	Posterior mean marker effects.
`dm`	Posterior inclusion probabilities or retained inclusion summaries.
`b`	Current or returned marker-effect state, when returned.
`d`	Current or returned inclusion state, when returned.
`r`	Marker-scale residual score state, when returned.
`vbs`	Retained trace of active marker-effect variance.
`vgs`	Retained trace of genetic variance.
`ves`	Retained trace of residual variance.
`pi`	Final or current inclusion-probability summary, when returned.
`pim`	Posterior mean inclusion-probability summary, when returned.
`input`	Resolved data, prior, scheduling, and MCMC settings recorded by the wrapper.

Depending on the implementation, fits can also contain `vb`, `vg`, `ve`, covariance summaries, LD-versus-linkage-equilibrium variance traces, or sampler diagnostics. Use model-specific help for their exact contract.

Compact reporting is preferable to printing complete marker matrices:

```
data.frame(
  pip_sum = sum(fit$dm[, 1]),
  mean_abs_bm = mean(abs(fit$bm[, 1])),
  mean_vb = mean(fit$vbs[, 1]),
  mean_vg = mean(fit$vgs[, 1]),
  mean_ve = mean(fit$ves[, 1])
)
```

Useful checks include trace behavior, repeated-chain agreement, sensitivity to priors and scheduling, posterior predictive behavior, top-marker stability, and BED-versus-CSR comparisons where both representations are available. Short workflow chains demonstrate integration; they are not sufficient for final inference. See Workflows and outputs for reporting guidance.

9 Transition to Annotation-Informed Priors

The plain ST-BLR model gives all markers a common active probability π and active-effect variance v_b. Annotation-informed extensions retain the same regression likelihood and marker-update logic while changing how prior mass or effect scale varies across markers.

9.1 Fixed Marker-Specific Priors

`stblr_csr_annot(annotation_model = "prior")` can replace the global values with fixed marker-specific inclusion probabilities π_i, variance multipliers, or both:

$$b_i \sim (1 - \pi_i)\delta_0 + \pi_i N(0, v_b \lambda_i).$$

In the marker update, π_i replaces the global prior inclusion probability and $v_b \lambda_i$ replaces the global active-effect variance.

9.2 Learned, Group, and Mixture Priors

- `stblr_csr_annot(annotation_model = "learned")` learns annotation effects on marker-specific inclusion probabilities and optionally variance multipliers.
- `stblr_csr_annot(annotation_model = "group")` gives mutually exclusive marker groups their own inclusion probabilities and variance multipliers.
- `stblr_csr_annot(annotation_model = "sbayesrc")` uses an **SBayesRC-style annotation-dependent BayesR prior**, assigning annotation-dependent probabilities to an exact-null component and multiple active variance components.

The SBayesRC-style wrapper is closely aligned with the published SBayesRC prior family but is not an exact reproduction of GCTB SBayesRC as exposed by default. Its default gamma grid and CSR sparse-LD likelihood differ; users can supply the published gamma grid explicitly when that prior grid is desired.

These extensions change the prior architecture, unlike sparse-update scheduling controls. Their definitions, assumptions, diagnostics, and interpretation cautions are developed in Annotation-informed models and Technical annotation priors.

10 Practical Model-Choice Summary

Need	Principal interface	Main consideration
Fit directly from available individual-level BED genotypes	`stblr_bed()`	Repeated BED access; no separately prepared sparse LD required.
Reuse sufficient statistics and disk-backed sparse LD	`stblr_csr()`	LD approximation, marker alignment, compatible cross-products, and optional `selection_s` settings are explicit inputs.
Reduce time spent repeatedly updating unlikely null markers	Scheduled BED or CSR path	Computational schedule must be validated for mixing and sensitivity.
Supply fixed marker-specific prior information	`stblr_csr_annot(annotation_model = "prior")`	Prior values are treated as fixed during fitting.
Learn annotation effects during fitting	`stblr_csr_annot(annotation_model = "learned")`	Additional parameters require regularization and convergence checks.
Use mutually exclusive category priors	`stblr_csr_annot(annotation_model = "group")`	Group assignment simplifies or discards overlap.
Use annotation-dependent null and multiple active components	`stblr_csr_annot(annotation_model = "sbayesrc")`	SBayesRC-style prior is more expressive and diagnostically demanding.

Whichever path is used, marker order, phenotype preprocessing, scaling, priors, MCMC settings, and computational controls should be recorded with the fit.

11 Related Notes and Sources

- Model overview
- Data representations
- Sparse-LD and BED workflows
- Annotation-informed models
- Workflows and outputs

- Technical summary statistics
- Technical sparse LD and CSR
- Technical annotation priors
- Complete sparse-LD/BED workflow on GitHub